

Calypso: SACMAT 2026 Artifact Documentation

Pankaj Niroula, Peyton Boggs, Aashutosh Poudel, Stephen Herwig

William & Mary, Department of Computer Science

{pniroula, prboggs, apoudel01, smherwig}@wm.edu

Corresponding author: Pankaj Niroula <pniroula@wm.edu>

April 18, 2026

1 Overview

This document accompanies the Calypso artifact submission to SACMAT 2026. It describes how to obtain, install, and exercise the artifact, and maps each artifact component to the corresponding paper claim.

Paper. *Calypso: Zero-Trust Service Discovery with Cryptographic Name Hiding* (SACMAT 2026).

Artifact repository. REPO_URL_PLACEHOLDER

Badges claimed. *ACM Artifacts Evaluated — Functional v1.1* and *ACM Artifacts Evaluated — Reusable v1.1*.

Reviewer time budget. A complete pass through the walkthrough in §6 takes approximately **60 minutes** of wall-clock time on the reference VM, including build.

2 What the Artifact Contains

Calypso extends zero-trust data-plane protections to cloud-based service discovery by hiding both DNS record contents and service names. The artifact ships the full implementation, the experiment orchestration used in the paper, and the paper’s raw data.

2.1 Source components (src/)

- **src/plugins/** — The three CoreDNS plugins authored for this paper: **jwt_edns** (JWT-over-OPT authorization), **etcd_crypto** (AES decryption for the enclave path and WKD-IBE), and **etcd_calypso** (search-tag-based routing for Calypso).
- **src/coredns-overlay/** — A small set of files copied or appended onto the upstream CoreDNS source tree at build time: **plugin.cfg** registrations, **Makefile** targets for SGX builds via *ego*, additional Go module requirements, *ego* enclave configurations, and a TLS certificate generator.
- **src/calypso/** — Core cryptographic Go package implementing WKD-IBE (built on Cloudflare’s *Circl* BLS12-381 pairing curve) and the Calypso construction (obfuscated names, reader/writer access controls, encrypt/decrypt/sign/verify). Source of the paper’s microbenchmarks.
- **src/q/** — Patched DNS query client used for client-side latency and packet-size measurements. Adds the **--measure-sizes** flag (paper Table 1) and granular per-stage timing in nanoseconds.

- `src/cryptofun/` — Go package providing baseline RSA-3072 benchmarks used as the comparison point in paper Table 2.
- `src/etcd-client/` — Record registration and key provisioning tool: populates DNS records into etcd and generates/derives WKD-IBE and Calypso keys.
- `src/jwt-tools/` — JWT generator (EdDSA, ES256, RS256) for the `jwt.edns` plugin.

The build pipeline (§5) clones upstream CoreDNS v1.12.4 and etcd into `experiments/repos/`, applies the overlay, and copies the plugins into the CoreDNS `plugin/` tree. The vendored `src/` directories themselves are not modified upstream code.

2.2 Experiment orchestration (`experiments/`)

Workspace for benchmarking, including the per-approach CoreDNS configurations, the latency-bench driver, and the plotting scripts. The `results/` subdirectory holds the paper run’s outputs: the baked CDF and percentile figures (PDF, EPS, PNG) are visible at the top, and the raw per-trial CSVs and aggregated summaries are bundled into `paper-run-data.zip`.

2.3 Helpers (`misc/`)

- `misc/gen-microbench-dat.py` — parses `go test -bench` output into the `.dat` format consumed by the gnuplot script that produces paper Figure 3.
- `misc/gen-rsa-comparison.py` — combines bench output from `src/calypso` and `src/cryptofun` to emit paper Table 2.
- `misc/measure-packet-sizes.sh` — per-approach driver that issues a single query through `q` and prints DNS request and response sizes.
- `misc/table1-reproduce.md` — step-by-step instructions for reproducing paper Table 1 with the script above.

3 Mapping to Paper Claims

Paper artifact	Reproduces	Section
Table 1 (packet expansion)	<code>misc/measure-packet-sizes.sh</code> (4 approaches); see	§5.1
Figure 3 (microbench crypto)	<code>misc/table1-reproduce.md</code> <code>src/calypso/ go test -bench=.</code> , parsed by	§5.1
Table 2 (Calypso vs. RSA-3072)	<code>misc/gen-microbench-dat.py</code> <code>src/cryptofun/ bench</code> compared via	§5.1
Figure 4 (E2E CDF, UDP)	<code>misc/gen-rsa-comparison.py</code> <code>experiments/run.sh compare plain-udp jwt-udp ...</code>	§5.2
Figure 5 (E2E CDF, DoH)	<code>experiments/run.sh compare plain jwt enclave-jwt</code> <code>wkdibe calypso</code>	§5.2
Figure 6 (latency percentiles)	<code>experiments/scripts/plot/plot-percentiles.py</code>	§5.2
Table 3 (latency breakdown)	Aggregated from per-approach summaries in <code>experiments/results/</code>	§5.2
Table 4 (server CPU overhead)	<code>experiments/scripts/collect-metrics.sh</code> then <code>analyze-metrics.sh</code>	§5.2

The paper’s section numbers above refer to the SACMAT 2026 final version.

4 Hardware and Software Requirements

Hardware. An Intel SGX-capable Linux VM is required to evaluate the full set of approaches. The paper uses an *Azure DC4s_v2* (4 vCPU, 16 GB RAM, Coffee Lake + SGX). Equivalent SKUs (DC4s_v3, etc.) are also acceptable. On non-SGX hardware, the two enclave-protected approaches will not start; the four other approaches (plain, jwt, wkdibe, calypso, plus their -udp variants) remain fully exercisable.

Operating system. Ubuntu 24.04 LTS (24.04.3 used in the paper run). Other Debian-derived distributions should work with the same package set.

Software. Installed automatically by the setup script: Go 1.25.3, GitHub CLI (gh), the ego framework (only when -with-ego is passed), Python 3 with matplotlib, gnuplot, and OpenSSL.

5 Installation

From the artifact root:

```
# 1. Install Go and create workspace dirs.
#    On an SGX-capable VM, append -with-ego to also install ego.
./experiments/scripts/setup-vm.sh -with-ego

# 2. Reload PATH in the current shell.
source ~/.bashrc

# 3. Stage vendored sources, clone upstream CoreDNS v1.12.4 + etcd,
#    and apply the overlay (registers our plugins, adds ego targets).
./experiments/scripts/use-vendored.sh

# 4. Build all components into experiments/build/.
#    -with-ego additionally produces an SGX-signed coredns binary.
./experiments/scripts/build-components.sh -with-ego
```

After this, the following binaries are present under `experiments/build/`: `coredns`, `etcd`, `etcdctl`, `etcd-client`, `jwt-tools`, `q`.

Verification. A single-query smoke test:

```
cd experiments
./run.sh test plain
```

should start CoreDNS+etcd, issue a DoH query for `a.default.svc.cluster.local`, print the resolved address, and stop the services cleanly.

6 Reviewer Walkthrough

This section lists the commands a reviewer should run to exercise the artifact end-to-end, with the expected observable outcome at each step. The repository's `docs/AE-WALKTHROUGH.md` carries a slightly more verbose version of the same tour.

Step 1 — Smoke test (~10 s)

```
cd experiments && ./run.sh test plain
```

Expected: the resolved IP is printed; services exit cleanly. Confirms the build is good and the simplest data path works.

Step 2 — Per-approach lifecycle (~2 min)

For each of `calypso`, `jwt`, `wkdibe`, and (on SGX) `enclave`:

```
cd experiments
./scripts/start.sh <approach>
./build/q -i TXT verify.example.com @https://localhost:8443
./scripts/stop.sh <approach>
```

Expected: each approach starts, returns a response, and stops. For `calypso`, the CoreDNS log (`approaches/calypso/coredns.log`) shows the EDNS option 65003 search-tag routing path described in §4 of the paper.

Step 3 — End-to-end latency comparison (~10 min)

Reproduces the *shape* of paper Figure 5 (DoH CDF):

```
./run.sh compare plain jwt wkdibe calypso 300 20 1
```

Expected: per-approach `*-aggregated-cdf.csv` files plus a combined CDF PDF, all written under `experiments/results/`. Compare the curve shapes against the paper’s baked plots in the same directory; absolute latencies will differ from the paper unless the run is on Azure DC4s_v2.

Step 4 — Packet expansion (paper Table 1) (~5 min)

See `misc/table1-reproduce.md` for the per-approach `q --measure-sizes` invocations. Each prints `dns_req=N dns_resp=M`; collating across the four approaches reproduces the columns of Table 1.

Step 5 — Microbench and RSA comparison (~15 min)

```
# Calypso operations vs. domain label count (paper Fig 3 source).
cd src/calypso
go test -bench=. -benchmem -run='^$' ../../ > /tmp/calypso-bench.txt
python3 ../../misc/gen-microbench-dat.py /tmp/calypso-bench.txt \
    > /tmp/calypso-bench.dat

# RSA-3072 baseline (paper Table 2).
cd ../cryptofun
go test -bench=. -benchmem -run='^$' ../../ > /tmp/rsa-bench.txt
python3 ../../misc/gen-rsa-comparison.py /tmp/calypso-bench.txt \
    /tmp/rsa-bench.txt
```

Expected: the microbench `.dat` shows monotonically increasing crypto cost with label count (matches Fig 3). The RSA comparison prints five rows whose ratio columns fall in the range of paper Table 2 (Encrypt 40–60×, Verify 30–45×).

Step 6 — Inspect baked results

Under `experiments/results/`, the combined CDF and percentile plots from the paper run are visible as `*.{pdf,eps,png}` files. The raw per-trial CSVs and aggregated summaries are bundled into `paper-run-data.zip` in the same directory; unzip in place to restore. The directory’s `README.md` carries the full inventory.

7 Limitations and Reviewer Notes

- **Reproducibility is not claimed.** SACMAT 2026 offers only the Functional and Reusable badges; absolute benchmark numbers will drift from the paper unless rerun on equivalent hardware (Azure DC4s_v2).
- **Network access required at first build.** `use-vendored.sh` clones upstream CoreDNS (v1.12.4) and etcd at their pinned versions; neither is bundled in `src/`.
- **Mixed licenses.** The artifact bundles BSD-3-Clause (our components), Apache-2.0 (CoreDNS), and GPL-3.0 (q upstream). All are mutually compatible for distribution; the inventory and compatibility note are in `LICENSES/README.md`.
- **Plot regeneration overwrites baked plots.** Re-running the pipeline will overwrite the paper's PDF/EPS in `experiments/results/`. Make a copy first if you want to diff before/after.